

Organizationally-Intelligent Agents: A Taxonomy and Characterization of Multi-Agent Engineering Systems Through Organizational Design Theory

Jessica Ezemba¹, Christopher McComb¹, and Conrad Tucker¹

Carnegie Mellon University, Pittsburgh PA, 15213, USA
jezemba@andrew.cmu.edu

Abstract. Organizational design has established principles for coordinating human teamwork over more than a century. Large language models have enabled multi-agent systems (MAS) for engineering design that invoke organizational terminology like "hierarchical" or "collaborative," yet current implementations fail to engage organizational design theory, making systematic comparison difficult. This paper characterizes thirty MAS frameworks using 8 attributes, then develops a taxonomy reducing organizational structures to three types (Orchestrated, Networked, Sequential) and operational methodologies to three patterns (Iterative-Feedback, Graph-Routed, Staged-Pipeline). Mapping these categories to organizational design theory, we demonstrate that each architecture invokes specific mechanisms: Orchestrated systems face span of control constraints, Networked systems require common ground establishment, and Sequential systems depend on modularity principles, yet current implementations engage these concepts without leveraging organizational research. Analysis reveals substantial fragmentation, with implementations ranging from 2 to 22 agents using various orchestration platforms. Our taxonomy provides foundations for systematic comparison and principled agent team design.

Keywords: Multi-agent Systems · Organizational Design · LLM Agents

1 Introduction

The design of modern complex engineered systems requires the coordinated efforts of multidisciplinary teams employing sophisticated organizational structures to manage complexity. Traditional engineering organizations use established frameworks like the V-Model to decompose tasks, allocate specialized roles, and coordinate across disciplines [1][2]. Additionally, emerging design theory frameworks such as Concept-Knowledge (C-K) theory [3], which characterizes design as operations between a concept space of undecided propositions and a knowledge space of established facts, provide new ways to characterize and structure the design process. Theories and patterns such as these reflect decades

of refinement in how human teams divide labor and integrate specialized knowledge.

Recently, large language models (LLM) have emerged as useful automata that can emulate human work. They have been used to create multi-agent systems (MAS) for engineering design, where LLM-based agents, acting as a design team, interact with domain-specific software to achieve high-level goals [4][5][6]. These MAS implementations frequently invoke organizational metaphors, characterizing themselves as "hierarchical" with manager agents coordinating specialists [7], "sequential" with stage-based workflows [8], or "collaborative" with peer-to-peer interaction [9]. However, these descriptors are inconsistently used. More fundamentally, current MAS research operates largely in isolation from organizational design (OD) theory.

Organizational design is a mature field with four major approaches: configuration (emphasizing fit and modularity), control (addressing span of control and oversight), channelization (directing attention and decision premises), and coordination (integrating effort through knowledge flows and common ground) [10]. This field of study has identified fundamental principles regarding modularity, span of control, decision rights allocation, and coordination mechanisms. Yet MAS implementations employ organizational labels without engaging these underlying principles, making *ad hoc* decisions about agent implementation rather than grounding those decisions in established frameworks. This gap prevents systematic comparison across systems and obscures whether organizational emulation enhances performance or constrains it. Systematic characterization and mapping efforts have proven valuable for understanding emerging computational paradigms; Kahlon and Fujii [11] demonstrated this approach by mapping design computing research according to researcher goals and supported design processes, providing the field with structured understanding of its landscape. However, to the best of the authors' knowledge, no comparable characterization exists that maps MAS organizational architectures to established organizational design theory.

This paper addresses these gaps through a systematic analysis of MAS frameworks from engineering design (2024-2025). We characterize each system using 8 attributes: orchestration frameworks, organizational structures, reasoning protocols, operational methodologies, state representations, modalities, agent populations, and coverage of Asimow's design activities [12]. From these attributes, we develop a 9-category taxonomy that reduces organizational structures to three mutually exclusive types (Orchestrated, Networked, Sequential) and operational methodologies to three patterns (Iterative-Feedback, Graph-Routed, Staged-Pipeline), enabling precise architectural comparison. We then map the shallow organizational terminology used in MAS literature to concepts from organizational design theory [10], revealing how current implementations engage superficially with organizational principles. This paper makes three primary contributions:

- We provide characterization and a principled taxonomy of MAS organizational architectures, enabling systematic comparison across systems.

- We map the taxonomy to organizational design theory, demonstrating that Orchestrated systems face span of control constraints, Networked systems require common ground establishment, and Sequential systems depend on modularity principles, yet current implementations do not explicitly engage these concepts.
- We develop a vocabulary grounded in organizational design concepts that can support more principled MAS architectures and enable the field to systematically explore how agent teams should be designed for engineering tasks.

Using organizational theory as a baseline in this way enables future research to identify both where MAS benefit from established human organizational patterns and where they require fundamentally different structures, advancing our understanding of what makes organizations effective across human and AI systems. The remainder of this paper is organized as shown in Figure 1.

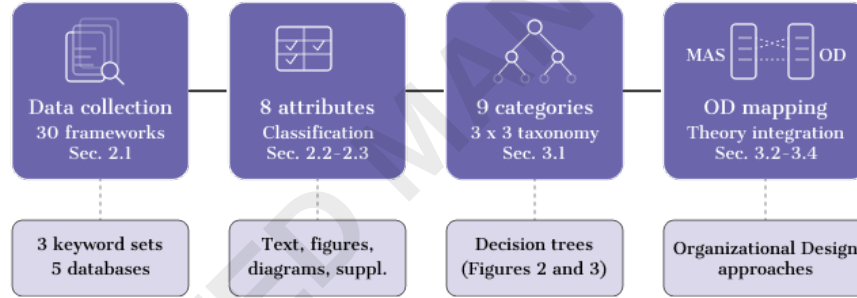


Fig. 1. Analytical pipeline from data collection through attribute identification, taxonomy development, and organizational design theory mapping

2 Multi-Agent Systems in Engineering Design

In this section we describe our data collection strategy for identifying representative MAS implementations, present 8 attributes capturing technical infrastructure choices, and report findings from analyzing thirty frameworks.

2.1 Methods: Data Collection and Classification

We included papers describing multi-agent systems employing LLMs or AI agents for hardware or physical product design (mechanical, structural, circuit, fluid mechanics, materials discovery) and we excluded software engineering (mature

agent-based tooling with distinct patterns), robotic RL agents (distinct action spaces and reward structures), and pure control theory or optimization without design synthesis. Searches employed three keyword sets ("Multi-Agent Framework for engineering design," "AI Agents in Engineering Design," "Agentic Frameworks for Engineering Design") targeting 2024-2025 publications across academic databases (arXiv, IEEE Xplore, ACM Digital Library), conference proceedings (Design Society, ASME IDETC-CIE), and engineering design venues (conducted November-December 2025). ArXiv preprints were included as cutting-edge LLM agent implementations often appear first in preprint form. This yielded thirty representative MAS frameworks spanning computer-aided design, structural analysis, circuit design, fluid mechanics, materials science, and multidisciplinary systems engineering.

2.2 Methods: Attributes of MAS Development

To enable systematic comparison of MAS organizational approaches, we identified 8 attributes capturing both technical infrastructure choices and organizational design decisions. Each attribute addresses a distinct dimension of the MAS architecture:

1. **Orchestration Framework** identifies the software platform coordinating agent interactions (AutoGen, LangChain, custom systems), reflecting infrastructure maturity and standardization.
2. **Organizational Structure** characterizes how agents are arranged and how authority and decision-making are distributed (hierarchical, sequential, collaborative, human-in-the-loop, etc.).
3. **Reasoning Protocol** identifies reasoning strategies (Chain-of-Thought (CoT), ReAct, Tree-of-Thoughts (ToT), Self-Reflection, Retrieval Augmented Generation (RAG), planning mechanisms), revealing how each agent handles reasoning complexity.
4. **Operational Methodologies** describes multi-agent operational logic governing task flow (closed control loops, supervisor-managed workflows, reflection loops, V-Model, Viable System Model (VSM), graph-based routing).
5. **State Representation** identifies how systems maintain information about design problems, partial solutions, and task progress (design state graphs, ChatML, JSON, persistent memory, vector databases).
6. **Modality** captures input/output types (text, PDF, Computer Aided Design (CAD), point clouds, simulation outputs, images), determining task breadth.
7. **Number of Agents** records agent count (from 2-agent pairs to 20+ ensembles), enabling investigation of relationships between agent count, task complexity, and organizational structure.
8. **Asimow Design Activity Coverage** identifies which recurring design activities the system addresses. Following Asimow's model [12], Analysis (requirements definition), Synthesis (artifact creation), and Evaluation (verification and validation) are treated as recurring and iterative activities rather than discrete stages, though for classification purposes we record which activities each framework engages.

To inform each attribute, we extracted information from paper text, figures, architectural diagrams, and supplementary materials. When papers did not explicitly report an attribute, we inferred it from described system behavior and diagrams, marking such cases with an asterisk (*) in Table 1. This notation allows readers to distinguish between author-reported and researcher-inferred classifications and to assess the strength of evidence underlying each characterization independently. This structured method enables systematic comparison of MAS organizational approaches across engineering domains.

2.3 Results: Attributes of Multi-Agent Systems

As shown in Table 1, the thirty frameworks span diverse engineering domains including computer-aided design, structural engineering, circuit design, fluid mechanics, and materials science. Orchestration platforms show concentration around established frameworks, with AutoGen (10 papers) and LangChain (4 papers) being most common, though 10 papers (33%) employ custom-built orchestration systems. Organizational structures are most frequently described as hierarchical, though 10 papers (33%) infer organizational structure from implementation rather than explicitly stating it. This inferential classification indicates significant variability in how transparently systems document their organizational design choices. Agent counts vary considerably: 3 frameworks employ 2-3 agents, 11 employ 4-5 agents, 3 employ 6-7 agents, 5 employ 8-11 agents, 2 employ 12+ agents, and notably, 6 frameworks use variable or task-dependent agent counts rather than fixed team sizes. ***This variation suggests no consensus on optimal team sizing for engineering design tasks***, contrasting with human engineering teams where 5-9 individuals is typically considered optimal [13].

Reasoning protocols show diversity in approaches, with CoT (8 papers), ReAct (8 papers), and RAG (8 papers) being commonly employed, often in combination. Task/Goal Decomposition (10 papers) and Planning Mechanisms (12 papers) appear frequently, reflecting engineering design's inherent complexity. Operational methodologies reveal patterns such as Iterative Feedback/Refinement Loops (10 papers), supervisor-managed workflows (5 papers), and closed-loop error refinement (4 papers). Only 1 framework explicitly implements V-Model architecture despite engineering design's historical reliance on this process framework. State representation approaches range from simple context window management to sophisticated design state graphs and persistent memory systems. Vector databases and embeddings appear in 6 frameworks, while structured memory (Core/Structured Memory, JSON storage, Knowledge Graphs) appears in 15 frameworks, indicating recognition that engineering design requires maintaining coherent representations of evolving artifacts. Modality support extends beyond text to include CAD files (6 papers), simulation outputs (9 papers), 3D geometry (6 papers), and code/scripts (11 papers), though text remains universal across all frameworks.

Coverage of Asimow's design activities reveals that 8 frameworks address Analysis only, 9 address Analysis-Synthesis, and 13 provide comprehensive Analysis-

Table 1. Comprehensive Overview of Multi-Agent Systems for Engineering Design

Dimension	Categories & Papers
SYSTEM ARCHITECTURE OVERVIEW	
Orchestration Framework	AutoGen [9], [14], [15], [16], [17], [18], [19], [20], [21], [22]; LangChain [23], [24], [25], [26]; LangGraph [27], [7]; CrewAI [28]; PydanticAI [29]; Camel AI [30]; ChatGLM [31]; Custom [32], [33], [34], [8], [35], [36], [37], [38], [39], [40]
Organization Structure	Hierarchical [34], [24], [15], [27] ^{*†} , [25], [18], [35] [†] , [29] [*] , [38], [28] [†] , [31] [†] ; Hierarchical+Collaborative [9], [33] [*] , [16] [*] , [30] [*] , [20] ^{*†} ; Collaborative [14], [17] [†] , [26], [36], [37], [21], [40] [†] , [7] [†] ; Sequential+Collaborative [32] ^{*†} , [8] [*] , [19]; Hybrid [22], [23] ^{*†} , [39] ^{*†}
Number of Agent	2-3 [23], [8], [26]; 4-5 [24], [35], [36], [29], [19], [37], [20], [38], [40], [22], [31]; 6-7 [33], [14], [28]; 8-11 [27], [18], [30], [21], [7]; 12+ [32], [39]; Variable [9], [34], [15], [16], [25], [17]
REASONING PROTOCOLS	
Reasoning	CoT [32], [16], [25], [18], [29], [30], [38], [39]; ReAct [15], [27], [17], [18], [36], [29], [30], [31]; ToT [23], [16], [18], [30]; RAG [32], [27], [16], [18], [35], [36], [19], [31]
Planning	Task Decomposition [9], [33], [14], [34], [15], [35], [36], [30], [38], [7]; Planning [33], [14], [8], [15], [17], [35], [26], [29], [19], [20], [38], [21]; Long-horizon [34], [18]; Memory Mgmt [34], [15], [18]
Refinement	Self-Reflection [32], [23], [24], [29], [37], [7], [28]; Self-Correction [9], [24], [36], [20], [28]; Reflexion [8], [17]
Specialized	Multimodal [24], [21], [22]; Physics-Aware [21]; Role-Playing [37], [39]; Optimization [40]
OPERATIONAL METHODOLOGIES	
Workflows	Iterative Refinement [32], [9], [33], [14], [34], [24], [17], [26], [38], [28]; Closed-Loop [32], [8], [17], [31]; Sequential [32], [16], [19], [31]; Graph-Based [15], [27], [7]; Design-Validation [8], [17], [18], [26], [36]
Architectures	FSM [20]; VSM [34]; V-Model [8]; Supervisor-Managed [24], [25], [35], [30], [7]
STATE REPRESENTATION	
Storage	Vector DB [9], [14], [26], [19], [21], [31]; Long-term [9], [23], [15], [7], [30]; Knowledge Graph [26], [37], [38], [24], [7]; Context Window [33], [24], [16], [17], [39]; Structured/JSON [14], [25], [18], [35], [29], [19], [7], [28], [32]; Other [20], [22], [40], [8], [36]; Stateless [27]
MODALITIES	
Primary	Text (all papers); Images [32], [9], [14], [8], [24], [17], [26], [36], [37], [20], [22], [31]; 3D Geometry [14], [8], [17], [19], [20], [22]; Code [9], [33], [23], [8], [18], [36], [19], [20], [38], [21], [7]
Specialized	Simulation Data [32], [23], [15], [16], [36], [29], [38], [28], [31]; PDF [32], [18], [26], [37], [30]; CAD Files [14], [8], [17], [19], [20], [22]; Netlists [16], [38]
ASIMOW DESIGN ACTIVITY	
Asimow Phase	Analysis Only [27], [25], [37], [30], [39], [40], [7], [28]; Analysis-Synthesis [23], [34], [8], [24], [15], [16], [19], [20], [38]; Full Pipeline (Analysis-Synthesis-Evaluation) [32], [9], [33], [14], [17], [18], [35], [26], [36], [29], [21], [22], [31]

^{*}Organizational structure inferred from implementation rather than explicitly stated

[†]System does not include human-in-the-loop interaction

Synthesis-Evaluation coverage. This distribution indicates that while most frameworks address problem formulation and artifact generation, fewer than half provide complete design cycle support including verification and validation. The gap in Evaluation coverage is particularly notable given that human engineering teams emphasize systematic testing protocols and formal review processes.

3 Organizationally-Intelligent Agent Taxonomy

Attribute analysis revealed inconsistent terminology and overlapping labels for organizational structures and operational methodologies, preventing systematic comparison across MAS implementations. We address this through taxonomy development, reducing descriptive complexity to two dimensions with three mutually exclusive categories each. This section presents the taxonomy development methodology, contextualizes categories against organizational design theory, reports the distribution across the 3×3 design space, and demonstrates how each architectural pattern inherently maps to specific organizational mechanisms.

3.1 Methods: Taxonomy Development

Our taxonomy focuses on two of the eight attributes: Organizational Structure and Operational Methodologies, as these exhibited the greatest ambiguity and required the most theoretical grounding. The remaining six attributes (orchestration framework, reasoning protocol, state representation, modality, agent count, and Asimow design activity coverage) did not require taxonomic reduction as their categories were sufficiently distinct across papers. Organizational structures were characterized using overlapping labels (“hierarchical,” “hierarchical + collaborative,” “collaborative,” “sequential + collaborative”), with 33% requiring inference from implementation details rather than explicit statement. Operational methodologies exhibited even greater diversity, with 17 distinct workflow patterns identified across the thirty representative frameworks. To address this limitation, we developed a two-dimensional taxonomy with mutually exclusive categories along each dimension, following the protocol-based classification approach of Hanna and Cagan [41] in which diverse implementations are reduced to a small set of distinct types.

Taxonomy Dimension 1: Organizational Structure We classify Organizational Structure based on the *coordination mechanism*; how the system allocates tasks and manages information flow between agents. We identified three mutually exclusive categories that capture distinct coordination patterns:

Orchestrated systems include an explicit orchestrator, manager, or supervisor agent whose primary role is coordinating other agents. The coordination mechanism involves centralized decision-making with delegated execution: the orchestrator routes tasks, controls information flow, and coordinates timing, while other agents execute tasks without making coordination decisions. Identifying features include explicit orchestrator/manager/supervisor/coordinator agents, hierarchical terminology with clear leader-subordinate relationships, and central control of task assignment and workflow sequencing. Examples include MASSE’s Management team directing junior engineers [18], Marco’s hierarchical collaboration framework [15], and GridMind’s agent coordinator managing inter-agent communication [29].

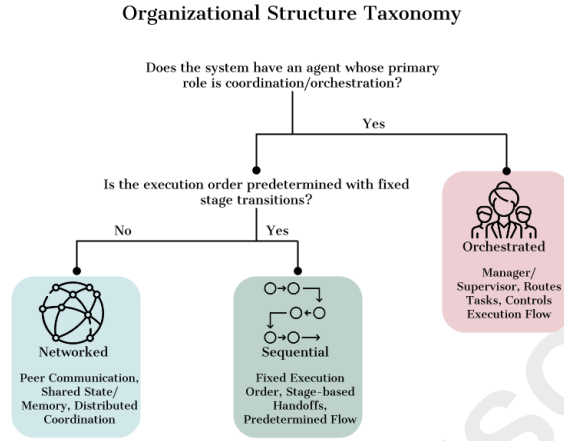


Fig. 2. Organizational Structure Taxonomy Decision Tree

Networked systems employ peer agents communicating through shared state, message passing, or blackboard architectures without a centralized coordinator. The coordination mechanism involves distributed coordination via shared memory/blackboard or direct agent-to-agent messaging, where each agent autonomously decides when and how to contribute, and coordination emerges from agent interactions rather than central control. Identifying features include group chat architectures, shared state or blackboard systems, peer-to-peer communication without mediator, collaborative terminology emphasizing equal partnership, and agents working “in tandem” or “autonomously collaborating.” Examples include MechAgents’ dynamical autonomous group chat [9], DesignGPT’s multi-agent role simulation [37], and AtomAgents’ autonomous collaboration [21].

Sequential systems implement pipeline or workflow architectures where agents execute in predetermined order with explicit stage-to-stage handoffs. The coordination mechanism involves fixed execution sequences with defined stage transitions, where agents activate in prescribed order, each stage completes before the next begins (though stages may iterate internally), and the emphasis is on workflow progression over coordination decisions. Identifying features include explicitly described stages, phases, or pipelines, linear workflow descriptions, V-Model implementations, and sequential task completion with handoffs. Examples include turbulence.ai’s three-stage architecture [32], AutoForma’s Requirement-Plan-Code-Execute pipeline [19], and V-Model implementations for CAD workflows [8]. Classification began by checking for a dedicated orchestration agent. A system not classified as Orchestrated (due to the absence of such an agent) was then evaluated based on its execution flow: it was classified as Sequential if the execution order was predetermined with fixed stage transitions, or as Networked otherwise as shown in Figure 2.

Taxonomy Dimension 2: Operational Methodologies We classify Operational Methodologies based on the *workflow control pattern*; what determines how work progresses through the system. Analysis revealed three distinct control patterns governing task execution:

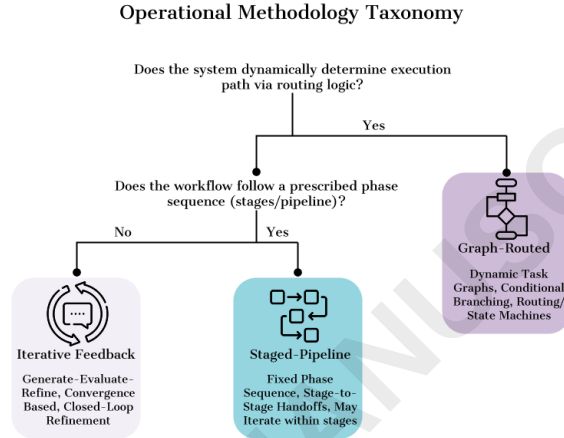


Fig. 3. Operational Methodologies Taxonomy Decision Tree

Iterative-Feedback systems employ repeated generation-evaluation-refinement cycles until convergence or quality thresholds are met. The control mechanism involves generating output, evaluating against criteria, refining based on feedback, and repeating until convergence or quality thresholds are satisfied. This represents convergence-based iteration with closed-loop refinement driven by quality assessment. Identifying features include explicit feedback loops, refinement cycles, or correction mechanisms; self-correction, reflection, or validation steps; iteration based on intermediate result quality; and terminology such as “closed-loop,” “refinement,” or “until convergence.” Examples include MechAgents’ self-correction loops [9], MASSE’s safety verification cycles [18], and closed-loop automotive styling design [17].

Graph-Routed systems employ dynamic task execution where the next step is determined by routing logic, state machines, or conditional branching rather than predetermined sequence. The control mechanism involves task graphs with nodes (sub-tasks) and edges (dependencies/conditions), where execution paths are determined dynamically based on current state, intermediate results, or classification/routing decisions, creating non-predetermined workflow topology. Identifying features include explicit graph structures for task representation, router agents or routing logic, state machines (FSM) or conditional branching, and dynamic path selection based on problem characteristics. Examples include

Marco’s configurable graph-based task solving [15], SLG’s orchestrator-expert routing [27], and foundation design’s router-based expert selection [25].

Staged-Pipeline systems implement fixed sequences of distinct phases with prescribed transitions between stages. The control mechanism involves predefined workflow phases executed in order, with stage-to-stage handoffs where output of stage N becomes input of stage $N + 1$. Phases may iterate internally but the overall sequence is predetermined, often mirroring engineering process models such as V-Model. Identifying features include named sequential stages or phases, pipeline terminology, engineering process model adoption, Chain-of-Stage reasoning, and explicit workflow phases in methods sections. Examples include V-Model implementations [8], turbulence.ai’s Idea-Simulation-Draft progression [32], and MenTeR’s Chain-of-Stage reasoning [16]. Classification began by determining if the system uses dynamic routing logic or state machines. A system not classified as Graph-Routed (due to the absence of dynamic execution path determination) was then evaluated based on its workflow structure: it was classified as Staged-Pipeline if it follows a prescribed phase sequence or pipeline, or as Iterative-Feedback otherwise as shown in Figure 3.

3.2 Methods: Contextualization Against Organizational Design Theory

To evaluate whether MAS implementations engage meaningfully with organizational design principles, we analyzed our 8 attribute results against the organizational design literature, primarily drawing on Joseph and Sengul’s comprehensive survey [10] reviewing research from 2000-2023 across four theoretical approaches: configuration, control, channelization, and coordination. Our analysis focused on two dimensions from our taxonomy: Organizational Structure and Operational Methodologies. We developed mapping tables connecting each taxonomy category to its most relevant organizational design concepts. This mapping process proceeded in three stages. First, for each taxonomy category, we identified the primary coordination or control challenge it addresses (e.g., Orchestrated systems face span of control limitations; Networked systems require common ground establishment). Second, we reviewed the four organizational design approaches to identify concepts directly addressing these challenges. Third, we prioritized concepts based on their centrality to each category’s defining mechanism, selecting the 3-6 most relevant OD concepts for each category rather than exhaustively listing all potentially related concepts. These mapping tables provide vocabulary for describing MAS architectures and enable assessment of which organizational design principles current implementations leverage or neglect.

3.3 Results: Taxonomy Distribution

Figure 4 shows the distribution of architectural patterns in current MAS practice. For Organizational Structure, **Orchestrated** architectures dominate (53%), followed by **Networked** (27%) and **Sequential** (20%). For Operational Methodologies, **Iterative-Feedback** is most common (50%), with **Staged-Pipeline**

(23%) and **Graph-Routed** (27%) less frequent. The combined design space shows clustering in specific cells: *Networked + Iterative-Feedback* (20%), *Orchestrated + Iterative-Feedback* (27%), *Sequential + Staged-Pipeline* (13%), and *Orchestrated + Staged-Pipeline* (10%). Notably, *Sequential + Networked* has zero implementations, which is theoretically consistent as sequential decomposition contradicts peer coordination requirements.

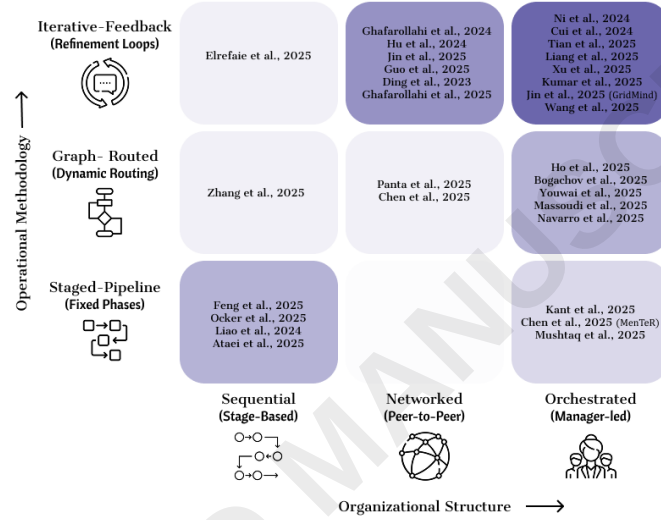


Fig. 4. 9-Category MAS Architecture Design Space

3.4 Mapping MAS Terminology to Organizational Design Concepts

The mapping between taxonomy categories and organizational design concepts (see supplementary materials) reveals that each architectural pattern engages a distinct subset of OD theory. For each category, we identify: (1) the primary OD approach(es) engaged, (2) the 5 most relevant OD concepts with operational definitions, and (3) why these concepts are essential rather than optional for that architectural pattern. **Orchestrated** systems primarily engage *Control and Coordination* approaches, as their manager-led structure inherently invokes span of control, oversight, and information asymmetry concepts. **Networked** systems primarily engage *Coordination and Channelization* approaches, as peer-to-peer coordination fundamentally depends on common ground, peer monitoring, and shared mental models. **Sequential** systems primarily engage *Configuration and Coordination* approaches, as stage-based decomposition inherently invokes modularity, mirroring hypothesis, and interface management concepts. Similarly, **Iterative-Feedback** workflows primarily engage *Channelization* concepts around aspiration levels and performance gaps, **Graph-Routed** workflows

engage *Channelization* concepts around attention allocation and routing logic, and **Staged-Pipeline** workflows engage *Configuration* concepts around decomposition and phase gates. Supplementary materials including the full corpus list, 8-attribute classifications, taxonomy rationale, the coding protocol, including attribute-specific extraction rules, decision tree application procedures, and selecting relevant OD concepts for each taxonomy category are documented in the supplementary materials <https://github.com/cmudrc/Supplementary-OIA>.

4 Discussion

The attribute analysis and taxonomy presented above expose both challenges and opportunities in current MAS practice for engineering design. While the field demonstrates innovation across diverse domains, fragmentation in terminology, orchestration approaches, and organizational structures prevents cumulative knowledge building. This section examines this fragmentation and its consequences, analyzes the architectural patterns and design space revealed by our taxonomy, evaluates how current implementations engage organizational design theory, and discusses implications for more principled MAS development. Where the following analysis moves beyond observed distributions to interpret why patterns emerge or predict how organizational mechanisms would operate, these interpretations represent testable hypotheses derived from organizational design theory rather than empirical findings of this study. Initial empirical evaluation of several such hypotheses, including the relationship between organizational structure choice and measurable coordination behavior, has been conducted in a companion study implementing all eight valid taxonomy combinations on a multidisciplinary aircraft design task [42].

4.1 Fragmentation and Limited Framework Adoption in Current Practice

Table 1 demonstrates considerable heterogeneity across all dimensions. Orchestration platforms range from AutoGen and LangChain to fully custom implementations (33%), with no dominant standard emerging. Agent counts vary from 2 to over 20, with 6 frameworks using variable team sizes and no clear relationship between task complexity and team configuration. ***Organizational structures predominantly combine hierarchical and collaborative elements, yet these labels are applied to systems with vastly different coordination mechanisms***, with 33% of structures inferred rather than explicitly stated. Cognitive protocols similarly span from simple CoT to sophisticated combinations of ReAct, ToT, and domain-specific reasoning.

This diversity reflects both rapid field evolution and domain-specific engineering challenges. Different disciplines (fluid mechanics, circuit design, structural analysis, materials science) have distinct toolchains and workflows necessitating customized architectures. However, this fragmentation creates challenges

for cumulative knowledge building. Without standardized vocabulary for organizational structures or common frameworks for comparison, identifying which architectural choices drive effectiveness, when particular patterns are appropriate, or how to build on prior successes becomes difficult. Terminological imprecision compounds this challenge. ***"Hierarchical" systems range from single manager agents coordinating specialists to multi-level hierarchies with intermediate coordinators. "Collaborative" systems include both peer-to-peer interaction and centrally-routed communication.*** This inconsistency makes systematic comparison difficult and obscures design decisions that organizational theory suggests have significant performance implications.

Despite targeting engineering design, few MAS implementations explicitly engage established process frameworks from human practice. ***Only 1 framework explicitly mirrors V-Model architecture,*** and few employ workflows aligning with Model-Based Systems Engineering (MBSE) principles. Most frameworks use *ad hoc* task decompositions based on functional requirements rather than leveraging systematic process structures refined over decades. Coverage of Asimow's design activities reveals that while most systems address Analysis and Synthesis, fewer than half provide comprehensive evaluation capabilities. This pattern suggests current implementations excel at requirements processing and artifact generation but struggle with validation and verification tasks that complete the engineering design cycle. From a C-K theory perspective, this distribution reveals which design operators current MAS implementations emphasize: Analysis, which involves $K \rightarrow C$ operations (transforming requirements into design concepts); Synthesis, which involves $C \rightarrow C$ and $K \rightarrow C$ operations (expanding concepts through partitioning and knowledge integration); and Evaluation, which involves $C \rightarrow K$ operations (validating concepts into verified knowledge). The limited Evaluation coverage suggests that while current systems excel at concept expansion and knowledge-driven synthesis, they underutilize the $C \rightarrow K$ conjunctions that transform exploratory concepts into validated design solutions. ***This pattern indicates that MAS implementations operate primarily in concept space, with limited mechanisms to systematically validate concepts back into the knowledge space.*** Human teams address this through systematic testing protocols and formal review processes, yet these mechanisms are rarely replicated in agent architectures. The limited adoption of engineering frameworks represents a missed opportunity, as the V-Model, MBSE, and related approaches encode accumulated knowledge about decomposing complex problems, managing interdependencies, ensuring traceability, and validating solutions.

4.2 Architectural Patterns and Design Space Revealed by Taxonomy

The taxonomy development addresses the fragmentation documented above by reducing descriptive labels to nine mutually exclusive categories with clear operational definitions. The taxonomy distribution, Figure 4, shows three dominant architectural patterns representing 64% of implementations. The most

common pattern is *Orchestrated + Iterative-Feedback* (27%) followed by *Networked + Iterative-Feedback* (20%), then *Orchestrated + Graph-Routed* (17%). This distribution suggests that current MAS practice favors flexibility and adaptability over predetermined structure. **Iterative-Feedback** methodologies dominate (50% overall) because engineering design inherently involves convergence to satisficing solutions rather than optimization to global optima. **Networked** and **Graph-Routed** architectures each represent 27% of implementations, while **Orchestrated** systems predominate at 53%, suggesting that manager-led coordination remains the dominant paradigm for engineering design MAS.

Figure 4 also shows architectural combinations that are rare or absent, representing potential research opportunities or fundamental incompatibilities. *Networked + Staged-Pipeline* has zero implementations, which is theoretically consistent: sequential decomposition into fixed stages contradicts peer coordination's requirement for flexible information sharing and emergent task allocation. The rigid stage boundaries that make **Staged-Pipeline** architectures tractable (each stage completes before the next begins) preclude the dynamic coordination that defines Networked systems. *Networked + Graph-Routed* has only two implementations (7%), representing an underexplored combination. This pattern would involve peer agents collectively determining execution paths through routing logic or state machines, potentially enabling adaptive workflows without central orchestration. The rarity suggests either implementation difficulty (coordinating peer routing decisions may require sophisticated consensus mechanisms) or conceptual unfamiliarity (graph-routed architectures are typically presented with centralized routers). This sparsity represents a research opportunity: Can peer systems effectively route attention collectively? What coordination mechanisms enable decentralized routing decisions?

Sequential + Graph-Routed also has only one implementation. This combination pairs predetermined stage sequences with dynamic path selection, which seems contradictory but could represent conditional stage activation based on intermediate results. The rarity suggests this hybrid approach has limited applicability or that implementations default to either fully sequential (**Staged-Pipeline**) or fully dynamic (**Graph-Routed**) rather than combining both principles. *Orchestrated + Graph-Routed* (17%) represents approximately one-third of **Orchestrated** systems. This pattern employs a manager agent with dynamic routing logic, which aligns with traditional organizational structures where managers allocate tasks based on current state. The moderate prevalence suggests this pattern is viable but may face complexity in implementing sophisticated routing logic or may be unnecessary for domains where fixed coordination patterns suffice.

4.3 Engagement with Organizational Design Theory

The taxonomy-based mapping (see supplementary materials) show gaps between how MAS implementations are designed and the theoretical depth organizational design research offers. Each architectural pattern identified in our taxonomy inherently invokes specific organizational design concepts, yet current implementa-

tions rarely engage these concepts explicitly. For example, of the sixteen papers classified as **Orchestrated**, none explicitly discuss span of control despite agent counts ranging from 4 to 11. Organizational design research consistently identifies span of control as a primary determinant of coordination cost in centralized structures [10], yet these papers select agent counts without reference to this principle. The gap is not that MAS papers fail to mention organizational principles in general, but rather that they fail to address the specific organizational challenges their chosen architectural patterns necessarily create. **Orchestrated** systems, which represent 53% of implementations, *unavoidably face span of control limitations; the number of subordinates a manager can effectively coordinate*. Human organizational research consistently finds optimal spans between 5-9 direct reports for coordination-intensive work [13], and our analysis shows **Orchestrated** systems range from 4-11 agents with a median of 8, suggesting implicit recognition of these constraints. These systems also inherently face information asymmetry, where the orchestrator must route tasks without possessing the detailed specialist knowledge of subordinate agents. This asymmetry is a fundamental feature of centralized coordination, not an implementation detail, yet it receives minimal attention in current literature.

Networked systems, representing 27% of implementations, fundamentally depend on common ground, shared knowledge that is known to be shared, to enable peer coordination without central control. Organizational research emphasizes that common ground establishment requires deliberate design: human teams invest significant effort developing shared mental models, terminology, and coordination protocols [10]. *Without deliberate common ground establishment, these systems risk either excessive communication overhead (agents constantly verifying mutual understanding) or coordination failures (agents assuming knowledge their peers lack)*. These systems could also face joint myopia risk, where shared information sources lead all agents toward the same suboptimal search patterns. **Sequential** systems, representing 20% of implementations, inherently invoke decomposition and modularity principles by breaking workflows into distinct stages. The effectiveness of such architectures depends critically on how stage boundaries are designed: *well-designed boundaries minimize interdependencies and enable parallel work within stages, while poorly designed boundaries create bottlenecks and cascading delays*. Papers could explicitly optimize stage boundaries using modularity principles, and discuss viscosity; the time required for changes in early stages to propagate through the pipeline.

This shallow engagement has practical consequences. Organizational research demonstrates that span of control, common ground establishment, and decomposition quality significantly impact coordination costs, information processing capacity, and decision quality. MAS designers who overlook these concepts may inadvertently recreate known organizational inefficiencies or miss opportunities to leverage established design principles. For example, the prevalence of *Networked + Iterative-Feedback* architectures (20%) suggests peer refinement is effective for design exploration, yet *without explicit common ground mechanisms*,

these systems may achieve coordination through inefficient trial-and-error rather than through principled design. The mapping also reveals opportunities for MAS to exploit organizational mechanisms particularly well-suited to AI agents. Common ground establishment, which requires substantial effort in human organizations, might be readily achieved through shared vector databases or synchronized knowledge bases in agent systems. While human organizations face fundamental span of control limits due to managerial attention constraints, AI orchestrator agents with appropriate information architectures might coordinate larger specialist teams (though determining these limits requires explicit investigation rather than assumption). Similarly, modularity principles suggest optimal decomposition strategies for **Sequential** systems, yet realizing these benefits requires deliberate application of organizational design theory rather than informal adoption of organizational metaphors.

4.4 Implications for MAS Design

This work contributes to the MAS literature by systematically mapping organizational terminology to established organizational design theory, providing readers with a shared vocabulary for comparing architectures across papers and a theoretical foundation identifying which organizational mechanisms each architectural choice inherently invokes. The mapping between MAS architectures and organizational design theory presented here enables two complementary research directions, both of which advance understanding regardless of outcome. If systematic application of organizational design principles improves MAS performance (for example, if deliberately designing for modularity reduces coordination costs or strategic choice of span of control enhances information processing) then the field gains actionable design guidance grounded in decades of organizational research. Alternatively, if effective MAS architectures systematically diverge from human organizational patterns, this divergence itself provides valuable insight. By establishing organizational theory as a well-understood baseline, we can identify precisely where and why AI systems require different structures; such as coordination mechanisms infeasible for humans (such as perfect common ground through shared memory architectures).

This does not imply agent systems should replicate human workflows; one central research question is identifying where AI-native organizational patterns outperform human-inspired structures. However, thoughtful engagement with established frameworks would enable more systematic evaluation of when emulation is beneficial and when divergence is warranted. Diversity in agent counts, reasoning protocols, and workflow patterns suggests important design trade-offs remain underexplored. Organizational design research has identified fundamental tensions such as specialization (which improves focus) versus integration (which manages interdependencies), or centralization (which enables firm-wide coordination) versus delegation (which leverages local information). Current MAS implementations make implicit choices about these trade-offs without systematic evaluation of alternatives. Future work could employ more rigorous experimental

designs to assess how organizational choices affect performance across different task types, complexity levels, and evaluation metrics.

The taxonomy presented here offers design researchers two immediate uses. First, it provides a reporting standard: researchers describing new MAS frameworks can classify their system along the two taxonomy dimensions, enabling direct comparison across papers using shared vocabulary rather than *ad hoc* labels. Second, it provides a design selection tool: researchers can use the taxonomy to reason about which architecture class suits their task profile before implementation. For instance, Ezemba et al. [42] used the taxonomy to systematically implement and compare all eight valid combinations on a shared engineering design task, finding that the choice of organizational structure and operational methodology produced distinct coordination behaviors that standard performance metrics alone could not distinguish.

4.5 Limitations and Future Directions

Our sample of thirty frameworks, while representing diverse engineering domains and recent implementations, is not exhaustive. The rapid pace of MAS development means new frameworks appear frequently, and as the field continues to grow, comprehensive literature reviews that capture the full breadth of MAS implementations across engineering domains will become both feasible and necessary. The taxonomy and mapping presented here provide a foundation for such efforts. Our analysis also focused on system descriptions in publications rather than empirical performance comparisons. Future work should extend characterization to larger samples, conduct empirical comparisons of different organizational approaches on standardized tasks. Longitudinal studies tracking how MAS organizational practices evolve would provide insight into whether convergence toward standard patterns emerges or domain-specific diversity persists. Most critically, experimental work is needed to test whether deeper engagement with organizational design principles (such as explicit design for modularity, strategic choice of span of control, or intentional establishment of common ground) improves MAS performance relative to current *ad hoc* approaches.

5 Conclusion

Multi-agent systems for engineering design operate largely in isolation from organizational design theory. This paper addresses this gap through identifying necessary attributes and taxonomy development. We identified 8 attributes for thirty representative MAS frameworks, then developed a 9-category taxonomy reducing organizational structures to three types (**Orchestrated**, **Networked**, **Sequential**) and operational methodologies to three patterns (**Iterative-Feedback**, **Graph-Routed**, **Staged-Pipeline**). These attributes reveals substantial fragmentation: 33% use custom orchestration, agent counts range from 2 to over 20, and 33% of organizational structures are inferred rather than stated. The taxonomy identifies three dominant architectural patterns and exposes theoretically

impossible combinations like *Networked + Staged-Pipeline* structures. Mapping taxonomy categories to organizational design theory demonstrates that each architecture inherently invokes specific mechanisms. **Orchestrated** systems face span of control constraints, **Networked** systems require common ground establishment, and **Sequential** systems depend on modularity principles, yet current implementations engage these concepts superficially. The path forward requires moving beyond informal organizational metaphors toward explicit engagement with organizational design principles. This does not imply agent systems should replicate human structures; understanding when to diverge remains essential. Rather, our taxonomy and organizational design mapping provide foundations for making design decisions systematically, enabling cumulative knowledge about what works, when, and why.

6 Acknowledgments

The authors gratefully acknowledge support from The Boeing Company and thank collaborators and engineers at Boeing for valuable discussions that informed this research. The views expressed in this article are those of the authors and do not necessarily reflect the views of The Boeing Company.

References

1. Iris Graessler and Julian Hentze. The new v-model of vdi 2206 and its validation. *at - Automatisierungstechnik*, 68(5):312–324, 2020.
2. Victor Vilhelm Poulsen, Matthias Guertler, Boris Eisenbart, and Nathalie Sick. Advancing systems engineering with artificial intelligence: a review on the future potential, challenges and pathways. *Proceedings of the Design Society*, 5:359–368, 2025.
3. Armand Hatchuel and Benoit Weil. A new approach of innovative design: an introduction to ck theory. In *DS 31: Proceedings of ICED 03, the 14th International Conference on Engineering Design, Stockholm*, 2003.
4. Alejandro Pradas Gomez, Maximilian Kretschmar, Kristin Paetzold-Byhain, and Ola Isaksson. A team of three: the role of generative ai in the development of design automation systems for complex products. *Proceedings of the Design Society*, 5:309–318, 2025.
5. Chijioke C Obieke, John Bridgeman, and Ji Han. A framework of ai collaboration in engineering design (aiced). *Proceedings of the Design Society*, 5:91–100, 2025.
6. Jerin George Mathew and Jacopo Rossi. Large language model agents. In *Engineering Information Systems with Large Language Models*, pages 173–205. Springer, 2025.
7. Soheyl Massoudi and Mark Fuge. Agentic large language models for conceptual systems engineering and design. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89237, page V03BT03A045. American Society of Mechanical Engineers, 2025.
8. Felix Ocker, Stefan Menzel, Ahmed Sadik, and Thiago Rios. From idea to cad: a language model-driven multi-agent system for collaborative design. *arXiv preprint arXiv:2503.04417*, 2025.

9. Bo Ni and Markus J Buehler. Mechagents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mechanics Letters*, 67:102131, 2024.
10. John Joseph and Metin Sengul. Organization design: Current insights and future research directions. *Journal of Management*, 51(1):249–308, 2025.
11. Yuval Kahlon and Haruyuki Fujii. Mapping design computing research. In *International Conference on Design Computing and Cognition*, pages 56–73. Springer, 2024.
12. Morris Asimow. Introduction to design. (*No Title*), 1962.
13. Kenny Nonso-Anyakwo and Joshua Summers. Impact of team size on requirement quantity. *Proceedings of the Design Society*, 5:2053–2061, 2025.
14. Alireza Ghafarollahi and Markus J Buehler. Rapid and automated alloy design with graph neural network-powered llm-driven multi-agent systems. *arXiv preprint arXiv:2410.13768*, 2024.
15. Chia-Tung Ho, Jing Gong, Yunsheng Bai, Chenhui Deng, Haoxing Ren, and Brucek Khailany. Marco: Configurable graph-based task solving and multi-ai agents framework for hardware design. *arXiv preprint arXiv:2504.01962*, 2025.
16. Pin-Han Chen, Yu-Sheng Lin, Wei-Cheng Lee, Tin-Yu Leu, Po-Hsiang Hsu, Anjana Dissanayake, Sungjin Oh, and Chinq-Shiun Chiu. Menter: A fully-automated multi-agent workflow for end-to-end rf/analog circuits netlist design. *arXiv preprint arXiv:2505.22990*, 2025.
17. Xinyu Jin, Shengmao Yan, Qingtao Wang, Shisong Deng, Yanzhen Jiang, and Shuangyao Zhao. A closed-loop multi-agent framework for aerodynamics-aware automotive styling design. *arXiv preprint arXiv:2508.03370*, 2025.
18. Haoran Liang, Yufa Zhou, Mohammad Talebi Kalaleh, and Qipei Mei. Automating structural engineering workflows with large language model agents. *arXiv preprint arXiv:2510.11004*, 2025.
19. JianXing Liao, JunYan Xu, SiCheng He, ZeKe Chen, Shui Yu, and Yun Li. Autoforma: A large language model-based multi-agent for computer-automated design. In *2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 1284–1289. IEEE, 2024.
20. Nirmal Panta, Saugat Kafley, Rujal Acharya, Sashank Parajuli, Dikshya Parajuli, Prince Panta, Saroj Belbase, Sudikshya Pant, Amit Regmi, Akio Tanaka, et al. Meda: A multi-agent system for parametric cad model creation. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89237, page V03BT03A042. American Society of Mechanical Engineers, 2025.
21. Alireza Ghafarollahi and Markus J Buehler. Automating alloy design and discovery with physics-aware multimodal multiagent ai. *Proceedings of the National Academy of Sciences*, 122(4):e2414074122, 2025.
22. Mohamed Elrefaie, Janet Qian, Raina Wu, Qian Chen, Angela Dai, and Faez Ahmed. Ai agents in engineering design: a multi-agent framework for aesthetic and aerodynamic car design. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89237, page V03BT03A048. American Society of Mechanical Engineers, 2025.
23. Bo Hu, Siyu Liu, Beilin Ye, Yun Hao, and Tongqi Wen. A multi-agent framework for materials laws discovery. *arXiv preprint arXiv:2411.16416*, 2024.
24. Jie Tian, Martin Taylor Sobczak, Dhanush Patil, Jixin Hou, Lin Pang, Arunachalam Ramanathan, Libin Yang, Xianyan Chen, Yuval Golan, Xiaoming Zhai, et al. A multi-agent framework integrating large language models and generative ai for accelerated metamaterial design. *arXiv preprint arXiv:2503.19889*, 2025.

25. Sompote Youwai, David Phim, Vianne Gayl Murcia, and Rianne Clair Onas. Investigating the potential of large language model-based router multi-agent architectures for foundation design automation: A task classification and expert selection study. *arXiv preprint arXiv:2506.13811*, 2025.
26. Varun Kumar and George Em Karniadakis. Toward autonomous engineering design: A knowledge-guided multi-agent framework. *arXiv preprint arXiv:2511.03179*, 2025.
27. Bogdan Bogachov and Yaoyao Fiona Zhao. A lightweight multi-expert generative language model system for engineering information and knowledge extraction. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89213, page V02BT02A030. American Society of Mechanical Engineers, 2025.
28. Tomas Navarro, Ana Stroescu, Dario Izzo, Sergio Gálvez Rojas, and Francisco López Valverde. Autonomous multi-agent ai systems for satellite mission design. In *2025 10th International Conference on Machine Learning Technologies (ICMLT)*, pages 366–372. IEEE, 2025.
29. Hongwei Jin, Kibaek Kim, and Jonghwan Kwon. Gridmind: Llms-powered agents for power system analysis and operations. In *Proceedings of the SC’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 560–568, 2025.
30. Abdullah Mushtaq, Rafay Naeem, Ibrahim Ghaznavi, Imran Taj, Imran Hashmi, and Junaid Qadir. Harnessing multi-agent llms for complex engineering problem-solving: A framework for senior design projects. In *2025 IEEE Global Engineering Education Conference (EDUCON)*, pages 1–10. IEEE, 2025.
31. Shijie Zhang, Xinrong Li, Chengxu Yuan, Wenqian Feng, and Quansheng Jiang. Desagent: A multi-agent mechanical design method based on collaborative large and small models. *Journal of Mechanical Design*, pages 1–46, 2025.
32. Jingsen Feng, Yupeng Qi, Ran Xu, Sandeep Pandey, and Xu Chu. turbulence. ai: an end-to-end ai scientist for fluid mechanics. *Theoretical and Applied Mechanics Letters*, page 100620, 2025.
33. Chenggang Cui, Jiaming Liu, Junkang Feng, Peifeng Hui, Amer MYM Ghias, and Chuanlin Zhang. Large language models based multi-agent framework for objective oriented control design in power electronics. *arXiv preprint arXiv:2406.12628*, 2024.
34. Neel Kant. Develop ai agents for system engineering in factorio. *arXiv preprint arXiv:2502.01492*, 2025.
35. Ran Xu, Yupeng Qi, Jingsen Feng, and Xu Chu. Engineering. ai: A platform for teams of ai engineers in computational design. *arXiv preprint arXiv:2511.00122*, 2025.
36. Bingkun Guo, Wentian Li, Xiaojian Liu, Jiaqi Luo, Zibin Yu, Dalong Dong, Shuyou Zhang, and Yiming Zhang. A multidisciplinary design and optimization (mdo) agent driven by large language models. *arXiv preprint arXiv:2511.17511*, 2025.
37. Shiyong Ding, Xinyi Chen, Yan Fang, Wenrui Liu, Yiwu Qiu, and Chunlei Chai. Designgpt: Multi-agent collaboration in design. In *2023 16th International Symposium on Computational Intelligence and Design (ISCID)*, pages 204–208. IEEE, 2023.
38. Zeyu Wang, Frank Po Wen Lo, Qian Chen, Yongqi Zhang, Chen Lin, Xu Chen, Zhenhua Yu, Alexander J Thompson, Eric M Yeatman, and Benny PL Lo. An llm-enabled multi-agent autonomous mechatronics design framework. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 4205–4215, 2025.

39. Mohammadmehdi Ataei, Hyunmin Cheong, Daniele Grandi, Ye Wang, Nigel Morris, and Alexander Tessier. Elicitron: A large language model agent-based simulation framework for design requirements elicitation. *Journal of Computing and Information Science in Engineering*, 25(2):021012, 2025.
40. Siyu Chen, Alparslan Emrah Bayrak, and Zhenghui Sha. A cost-aware multi-agent system for black-box design space exploration. *Journal of Mechanical Design*, 147(1):011703, 2025.
41. Lindsay Hanna Landry and Jonathan Cagan. Protocol-based multi-agent systems: examining the effect of diversity, dynamism, and cooperation in heuristic optimization approaches. 2011.
42. Jessica Ezemba, Christopher McComb, and Conrad Tucker. Structure shapes coordination: An empirical evaluation of organizational design principles in multi-agent engineering design. In *In Review*, 2026.